

Ex. No: 9 Threading

C PROGRAM

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>

void *thread_function(void *arg)

{

    int i;

    for(i = 1; i <= 5; i++)

    {

        printf("Thread Executing : %d\n", i);

        sleep(1);

    }

    pthread_exit(NULL);

}

int main()

{

    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_function, NULL);

    pthread_create(&t2, NULL, thread_function, NULL);

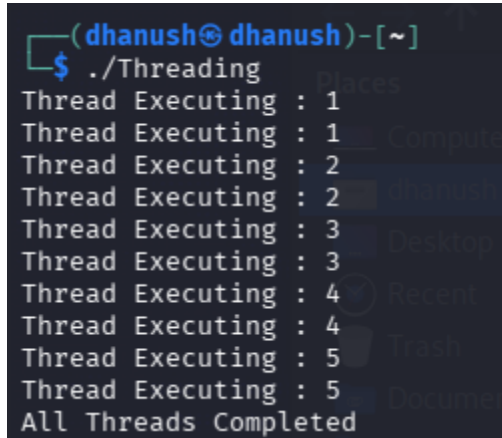
    pthread_join(t1, NULL);

    pthread_join(t2, NULL);

    printf("All Threads Completed\n");
```

```
return 0;
}
```

Output:

A terminal window with a dark background. The prompt is '(dhanush@dhanush)-[~]'. The command './Threading' has been executed. The output consists of ten lines: 'Thread Executing : 1', 'Thread Executing : 1', 'Thread Executing : 2', 'Thread Executing : 2', 'Thread Executing : 3', 'Thread Executing : 3', 'Thread Executing : 4', 'Thread Executing : 4', 'Thread Executing : 5', and 'Thread Executing : 5'. The final line is 'All Threads Completed'.

```
(dhanush@dhanush)-[~]
$ ./Threading
Thread Executing : 1
Thread Executing : 1
Thread Executing : 2
Thread Executing : 2
Thread Executing : 3
Thread Executing : 3
Thread Executing : 4
Thread Executing : 4
Thread Executing : 5
Thread Executing : 5
All Threads Completed
```

SHELL SCRIPT

Shell scripting does not support threads directly like C. However, concurrent execution can be simulated using background processes (&).

```
#!/bin/bash
```

```
task1()
```

```
{
```

```
for i in 1 2 3 4 5
```

```
do
```

```
echo "Thread 1 : $i"
```

```
sleep 1
```

```
done
```

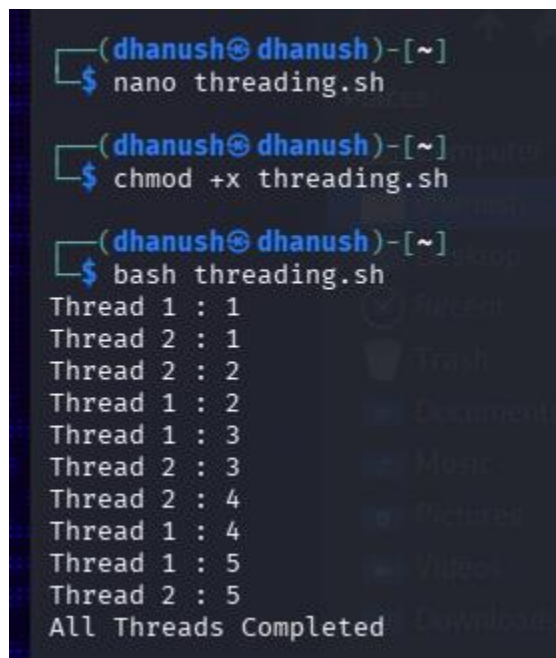
```
}
```

```
task2()
```

```
{
```

```
for i in 1 2 3 4 5
do
echo "Thread 2 : $i"
sleep 1
done
}
task1 &
task2 &
wait
echo "All Threads Completed"
```

Output:



```
(dhanush@dhanush)-[~]
$ nano threading.sh
(dhanush@dhanush)-[~]
$ chmod +x threading.sh
(dhanush@dhanush)-[~]
$ bash threading.sh
Thread 1 : 1
Thread 2 : 1
Thread 2 : 2
Thread 1 : 2
Thread 1 : 3
Thread 2 : 3
Thread 2 : 4
Thread 1 : 4
Thread 1 : 5
Thread 2 : 5
All Threads Completed
```

The screenshot shows a terminal window with a dark background. The prompt is (dhanush@dhanush)-[~]. The user runs 'nano threading.sh', then 'chmod +x threading.sh', and finally 'bash threading.sh'. The output shows two parallel threads, Thread 1 and Thread 2, each executing a loop from 1 to 5. Thread 1 prints its values first, followed by Thread 2. The final output is 'All Threads Completed'.